DEVOPS REHBERI

Yeni Baslayanlar Icin Docker El Kitabi

Konteyner teknolojilerine adim atanlar icin kapsamli baslangic rehberi

Konular: Docker Mimarisi, Temel Komutlar, Dockerfile

2026

Icindekiler

1. Docker Nedir?

1.1 Konteynerlestirme Kavrami

1.2 Neden Docker Kullanmaliyiz?

1.3 Docker'in Tarihcesi

2. Docker ve Sanal Makine Karsilastirmasi

2.1 Sanal Makine (VM) Nedir?

2.2 VM ile Docker Arasindaki Temel Farklar

2.3 Ne Zaman VM, Ne Zaman Docker?

3. Docker Mimarisi

3.1 Docker Engine

3.2 Docker Image (Goruntu)

3.3 Docker Container (Konteyner)

3.4 Docker Hub ve Kayit Defterleri

3.5 Docker Mimarisi Diyagrami

4. Temel Docker Komutlari

4.1 Konteyner Yonetim Komutlari

4.2 Image Yonetim Komutlari

4.3 Ag ve Volume Komutlari

4.4 Docker Compose Temelleri

5. Dockerfile ile Uygulama Olusturma

5.1 Dockerfile Nedir?

5.2 Nginx Web Sunucusu Ornegi

5.3 Python Web Uygulamasi Ornegi

5.4 Multi-Stage Build

6. Sonuc ve Oneriler

1. Docker Nedir?

> *Bu bolumde Docker'in temel tanimini, konteynerlestirme kavramini ve bu teknolojiyi ogrenmenin neden bu kadar onemli oldugunu ele alacagiz.*

1.1 Konteynerlestirme Kavrami

Docker, uygulamalarin ve bunlarin calismasi icin gereken tum bagimliliklarin (kutuphaneler, sistem araclari, kod ve ayar dosyalari) bir araya getirilerek **standartlastirilmis birimler** halinde paketlenmesini saglayan bir platformdur. Bu paketlere **konteyner (container)** adi verilir.

Konteynerlestirme, bir uygulamanin gelistirme ortaminda nasil calisiyorsa, test ve uretim ortamlarinda da **tam olarak ayni sekilde calismasini garanti eden** bir teknolojidir. "Benim bilgisayarimda calisiyordu" problemini kokunden cozer.

> **Docker'in Resmi Tanimi**
>
> Docker, gelistiricilerin uygulamalarini altyapi detaylarindan ayirarak hizli bir sekilde gelistirmesine, dagitmasina ve calistirmasina olanak tanimak uzere tasarlanmis acik kaynakli bir platformdur.

Konteyner teknolojisinin ana fikri soyledir: Bir uygulama ve calismasi icin gerekli olan her sey (isletim sistemi kutuphaneleri, calisma zamanlari, sistem araclari) tek bir kapsul icerisinde birlestirilir. Bu kapsul, herhangi bir Docker kurulu olan sistemde tutarli bir sekilde calisir.

1.2 Neden Docker Kullanmaliyiz?

Docker, modern yazilim gelistirme sureclerinde vazgecilmez bir arac haline gelmistir. Iste Docker kullanmanin en onemli nedenleri:

Tutarlilik (Consistency)

Gelistirme, test ve uretim ortamlari arasindaki farkliliklar yazilim hatalarinin baslica kaynagidir. Docker ile uygulamanizin calisma ortami bir imaj (image) olarak paketlenir ve bu imaj her ortamda **ayni sekilde calisir**. "Benim bilgisayarimda calisiyordu" denen bir durum bir daha yasanmaz.

Hafiflik (Lightweight)

Sanal makinelere kiyasla konteynerler cok daha az kaynak tuketir. Bir konteyner, ana isletim sisteminin cekirdegini paylasir ve sadece uygulama icin gerekli olan kutuphaneleri icerir. Bu sayede **tek bir fiziksel sunucuda onlarca hatta yuzlerce konteyner** calistirilabilir.

Hizli Dagitim (Fast Deployment)

Konteynerler saniyeler icinde baslatilabilir. Bu, uygulamalarin hizli bir sekilde olceklendirilmesini ve guncellenmesini saglar. Yeni bir surum dagitmak, eski surumu durdurup yenisini baslatmak kadar kolaydir.

Izolasyon (Isolation)

Her konteyner kendi dosya sistemi, ag arayuzu ve calisma ortamina sahiptir. Bir konteynerdeki bir sorun diger konteynerleri etkilemez. Bu, ayni sunucuda farkli uygulamalarin guvenli bir sekilde yan yana calismasini saglar.

Portabilite (Portability)

Docker imajlari, Docker'in calistigi herhangi bir sistemde calisir. Bu, gelistiricinin dizustu bilgisayarinda olusturulan bir imajin bulut sunucularinda, fiziksel sunucularda veya hibrit ortamlarda sorunsuz calisacagi anlamina gelir.

Tablo 1 Docker Kullanim Avantajlari

Ozellik	Geleneksel Yontem	Docker ile
Ortam hazirlama suresi	Saatler - Gunler	Dakikalar
Kaynak kullanimi	Yuksek (her uygulama icin ayri OS)	Dusuk (OS cekirdegini paylasma)
Ortam tutarliligi	Dusuk (her ortam farkli)	Yuksek (ayni imaj her yerde)
Olcaklendirme	Yavas (dakikalar)	Hizli (saniyeler)
Uygulama izolasyonu	Zor ve maliyetli	Dogal ve kolay

1.3 Docker'in Tarihcesi

Docker, 2013 yilinda Solomon Hykes tarafindan **DotCloud** adli bir Platform as a Service (PaaS) sirketi icinde bir ic proje olarak baslatildi. Ilk olarak Python ile yazilan Docker, daha sonra Go programlama diline gecirildi.

2013 yilinin Mart ayinda PyCon konferansinda gosterimi yapilan Docker, endustri uzerinde ani bir etki yaratti. Konteyner teknolojisi aslinda yeni degildi - Linux cekirdegindeki LXC (Linux Containers) teknolojisi uzun suredir mevcuttu. Ancak Docker, bu teknolojiyi kullanici dostu bir arayuzle birlestirerek **konteynerlerin kitlesel olarak benimsenmesini sagladi**.

Bugun Docker, milyonlarca gelistirici tarafindan kullanilan ve yazilim endustrisinde bir standart haline gelmis acik kaynakli bir projedir.

2. Docker ve Sanal Makine Karsilastirmasi

> *Docker'i tam olarak anlamak icin oncelikle geleneksel sanallastirma teknolojisi olan Sanal Makineleri (VM) anlamak gerekir. Bu bolumde iki teknolojiyi karsilastiracagiz.*

2.1 Sanal Makine (VM) Nedir?

Sanal Makine (Virtual Machine - VM), fiziksel bir sunucunun donanimini sanal olarak bolen ve her bolumun kendi isletim sistemini calistirmasina olanak tanimlayan bir teknolojidir. VM'lerin calismasi icin **Hipervizor (Hypervisor)** adi verilen bir yazilima ihtiyac vardir.

Hipervizorler iki turdedir:

- **Tur 1 (Bare-Metal):** Dogrudan fiziksel donanim uzerinde calisir. Ornek: VMware ESXi, Microsoft Hyper-V, Xen
- **Tur 2 (Hosted):** Ana isletim sistemi uzerinde calisir. Ornek: VMware Workstation, Oracle VirtualBox

Her sanal makine, kendi isletim sistemi cekirdegini, sistem kutuphanelerini ve uygulamalarini iceren tam bagimsiz bir ortamdir. Bu, izolasyon acisindan guclu bir cozum sunar ancak kaynak kullanimi acisindan maliyetlidir.

2.2 VM ile Docker Arasindaki Temel Farklar

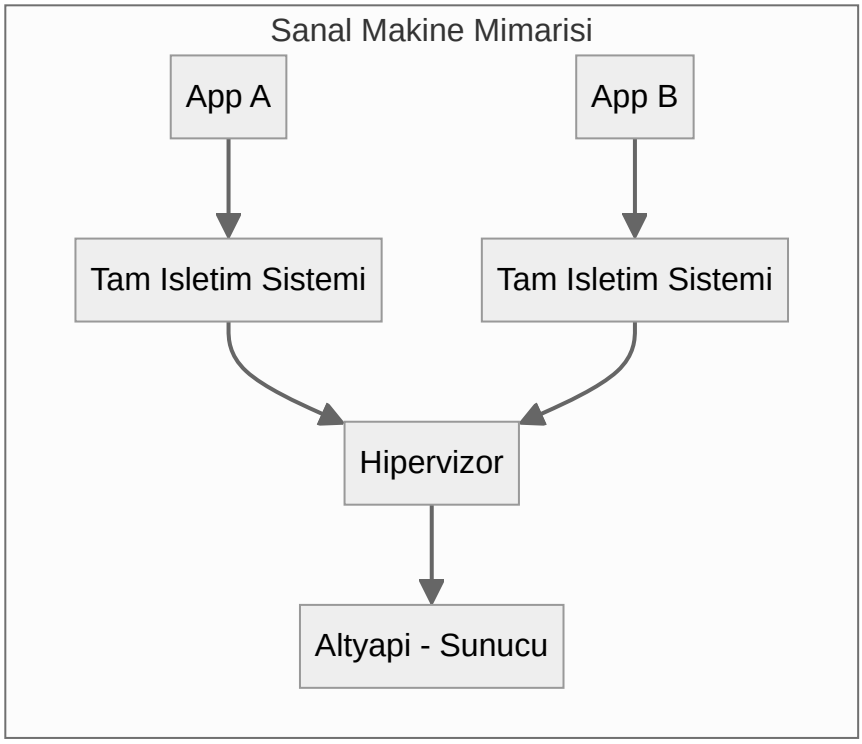
Docker konteynerleri ile sanal makineler arasindaki temel fark, **isletim sistemi katmaninin paylasilip paylasilmadigidir**.

Tablo 2 Docker Konteynerleri ve Sanal Makineler Karsilastirmasi

Ozellik	Sanal Makine (VM)	Docker Konteyneri
Isletim Sistemi	Her VM'de ayri bir OS cekirdegi	Ana OS cekirdegini paylasir
Boyut	Sabit disk: GB'larca (5-20 GB)	Image: MB'larca (10-500 MB)
Baslangic suresi	Dakikalar (OS acilmasi gerekir)	Saniyeler
Kaynak kullanimi	Yuksek (ayri RAM, CPU atamasi)	Dusuk (ana OS'den paylasimli)

Performans	Biraz dusuk (sanallastirma katmani)	Yakin-native (minimal ek yuk)
Izolasyon	Tam donanim duzeyinde izolasyon	Surec duzeyinde izolasyon
Olceklendirme	Zor ve yavas	Kolay ve hizli
Tasinabilirlik	Image dosyalari buyuk ve agir	Image'lar kucuk ve hafif

Bu farkliligi daha iyi anlamak icin mimari yapiya bakalim:



Sekil 1 Sanal Makine mimarisinde her uygulama icin ayri isletim sistemi gerekir

Bellek Kullanimi Ornegi

Ornegin, 3 farkli uygulama calistirmak istediginizi dusunun. Her biri Ubuntu isletim sistemi gerektiriyor:

- **VM ile:** 3 x Ubuntu (her biri ~1 GB RAM) = 3 GB sadece isletim sistemleri icin
- **Docker ile:** 1 x Ubuntu (paylasilan) + 3 konteyner = Cok daha az bellek kullanimi

2.3 Ne Zaman VM, Ne Zaman Docker?

Her iki teknolojinin de kendine ozgu avantajlari vardir. Dogru secim, kullanim senaryonuza baglidir:

Docker Kullanin:

- Mikroservis mimarisi ile calisiyorsaniz
- Hizli olceklendirme ve dagitim ihtiyaciniz varsa
- Surekli entegrasyon / surekli dagitim (CI/CD) pipeline'i kuruyorsaniz
- Farkli ortamlarda tutarli calisma istiyorsaniz
- Kaynak verimliligi onemliyse

Sanal Makine Kullanin:

- Farkli isletim sistemlerini ayni anda calistirmak istiyorsaniz (ornegin Windows uzerinde Linux)
- Tam donanim duzeyinde izolasyon gerekiyorsa
- Guvenlik gereksinimleri en ust duzeydeyse
- Legacy uygulamalari calistirmak icin
- Guclu ve tamamen yonetilebilir bir ortam istiyorsaniz

Hybrid Yaklasim

Gercek dunya senaryolarinda genellikle her iki teknoloji birlikte kullanilir. Ornegin, bir bulut saglayicisinda VM'ler uzerinde Docker konteynerleri calistirabilirsiniz. Bu "VM'lerin icinde konteynerler" yaklasimi, hem guclu izolasyon hem de konteyner esnekligini bir arada sunar.

3. Docker Mimarisi

> *Docker'in nasil calistigini anlamak icin temel bilesenlerini ogrenmemiz gerekir. Bu bolumde Docker Engine, Image, Container ve Docker Hub gibi temel kavramlari detayli olarak inceleyecegiz.*

3.1 Docker Engine

Docker Engine, konteynerleri olusturmak ve calistirmak icin kullanilan Docker'in temel motorudur. Istemci-sunucu (client-server) mimarisine sahiptir ve uc ana bilesenden olusur:

Docker Daemon (dockerd)

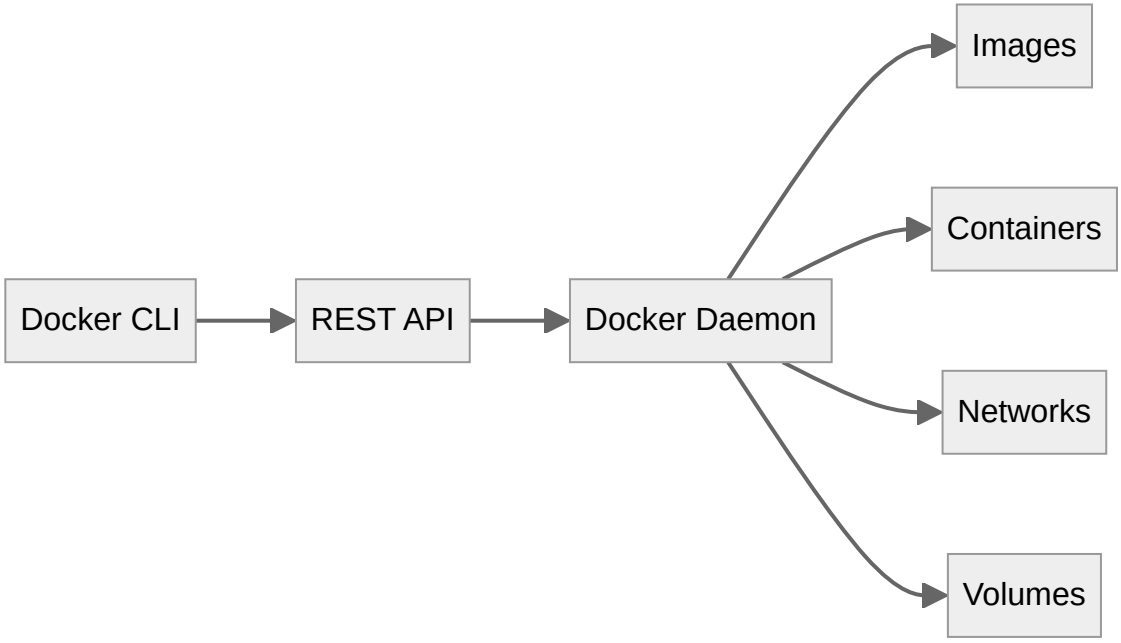
Arka planda calisan sunucu surecidir. Docker objelerini (image, container, network, volume) yonetmekten sorumludur. REST API uzerinden diger bilesenlerle iletisim kurar.

Docker API

Daemon ile diger bilesenler arasindaki iletisimin saglandigi arayuzdur. RESTful API mimarisi kullanir.

Docker CLI (docker)

Kullanicinin komut satirindan Docker ile etkilesime girdigi aractir. Yazdiginiz komutlar CLI araciligiyla API'ye, oradan da Daemon'a iletilir.



Sekil 3 Docker Engine bilesenleri arasindaki iletisim

3.2 Docker Image (Goruntu)

Docker Image, bir konteyner olusturmak icin kullanilan salt okunur (read-only) sablonlardir. Bir image, bir uygulamayi calistirmak icin gereken her seyi icerir: kod, calisma zamanlari (runtime), kutuphaneler, ortam degiskenleri ve konfigurasyon dosyalari.

Image'lar **katmanli (layered)** bir yapida olusturulur. Her katman, bir onceki katman uzerine eklenen degisiklikleri temsil eder. Bu katmanli yapi sayesinde:

- Ayni katmanlar birden fazla image arasinda **paylasilabilir**
- Image boyutlari optimize edilir
- Image olusturma sureci hizlanir

> **Katmanli Yapi Ornegi**
>
> Bir Python uygulamasi icin image katmanlari soyle gorunebilir:
>
> `Katman 1:` Ubuntu temel image (120 MB)
> `Katman 2:` Python kutuphanelerinin eklenmesi (45 MB)

Temel Image Komutlari

Tablo 3 Image Islemleri Icin Temel Komutlar

Komut	Aciklama	Ornek Kullanim
`docker images`	Yerel imaj listesini gosterir	`docker images`
`docker pull`	Kayittan imaj indirir	`docker pull nginx:latest`
`docker rmi`	Imaji siler	`docker rmi nginx:latest`
`docker build`	Dockerfile'dan imaj olusturur	`docker build -t myapp:1.0 .`
`docker history`	Imaj katmanlarini gosterir	`docker history nginx`

3.3 Docker Container (Konteyner)

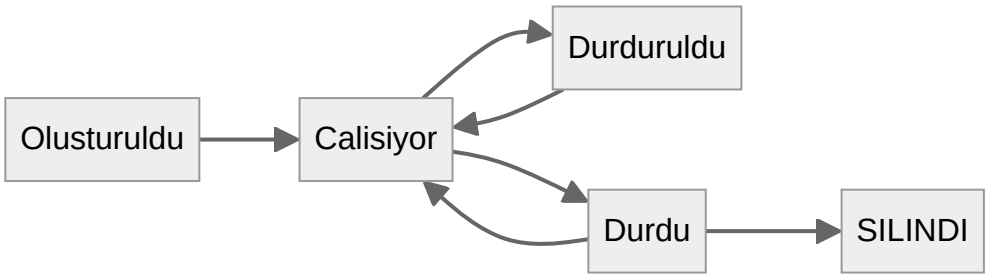
Docker Container, bir Docker Image'in calisan ornegidir (instance). Image salt okunur bir sablonken, container bu sablonun calisan, yazilabilir hale gelmis halidir. Bir container calistirildiginda, image'in uzerine yazilabilir bir katman eklenir.

Container'larin temel ozellikleri:

- **Yalitilmis:** Her container kendi dosya sistemi, ag baglantisi ve calisma ortamina sahiptir
- **Tasinabilir:** Herhangi bir Docker kurulu sistemde calisir
- **Kontrollu:** CPU ve bellek limitleri atanabilir
- **Gecici:** Varsayilan olarak kalici depolama yapmaz (volume kullanilmadiginda)

Container Yasam Dongusu

Container'lar belirli bir yasam dongusu icinde calisirlar:



Sekil 4 Bir konteynerin yasam dongusu

3.4 Docker Hub ve Kayit Defterleri

Docker Hub, Docker tarafindan isletilen bir bulut tabanli kayit defteridir (container registry). Milyonlarca public Docker Image'in barindigi dunyanin en buyuk container kütüphanesidir.

Docker Hub uzerinde:

- **Resmi Image'lar:** Docker tarafindan dogrulanmis, guvenli image'lar (nginx, python, mysql, redis vb.)
- **Topluluk Image'lari:** Kullanicilar tarafindan paylasilan image'lar
- **Ozel Depolar:** Kendi image'larinizi ozel olarak saklayabilirsiniz

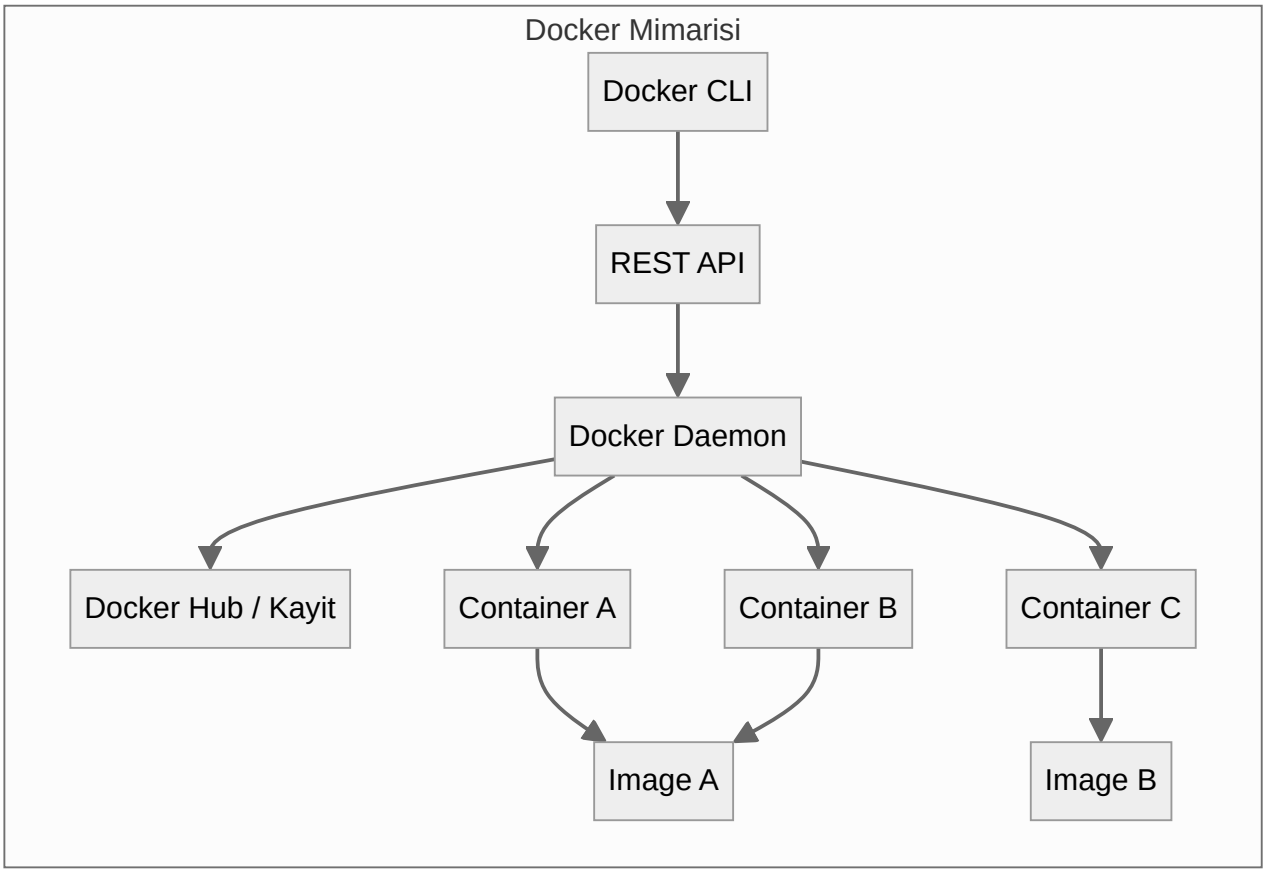
Docker Hub disinda alternatif kayit defterleri de vardir:

- **Amazon ECR** (Elastic Container Registry)
- **Google GCR** (Google Container Registry)
- **Azure ACR** (Azure Container Registry)
- **GitHub Container Registry**
- **Harbor** (Acik kaynakli, kendi sunucunuzda calistirabilirsiniz)

> **Docker Hub Kullanim Ipuclari**
>
> Resmi image'lari kullanmak her zaman daha guvenlidir. Resmi image'lar `library/` on ekine sahiptir ve genellikle cekirdek kutuphane veya uygulamalardir. Ornegin `docker pull nginx` komutu aslinda `docker pull library/nginx` demektir.

3.5 Docker Mimarisi Diyagrami

Docker'in tum bilesenlerini bir arada gorelim:



Sekil 5 Docker ekosisteminin genel mimarisi

Tablo 4 Docker Mimarisi Bilesenleri Ozeti

Bilesen	Islev	Analogi
Docker Engine	Konteynerleri olusturan ve calistiran motor	Bir fabrikanin ana makinesi
Docker Image	Salt okunur sablon, konteyner olusturmak icin	Bir fabrikadaki kalip
Docker Container	Image'in calisan ornegi	Kaliptan uretilen urun
Dockerfile	Image olusturmak icin talimat dosyasi	Uretim talimati
Docker Hub	Image'larin depolandigi bulut servisi	Urunlerin satildigi magaza

4. Temel Docker Komutlari

> *Bu bolumde Docker ile calismak icin bilmeniz gereken en onemli komutlari ogreneceksiniz. Bu komutlari gunluk kullanimda siklikla ihtiyac duyacaksiniz.*

4.1 Konteyner Yonetim Komutlari

Konteyner olusturma, baslatma, durdurma ve yonetme islemleri icin temel komutlar:

Konteyner Olusturma ve Calistirma

```
# Yeni bir konteyner olustur ve calistir
docker run nginx

# Arka planda calistir (detached mode)
docker run -d nginx

# Bir isim vererek calistir
docker run --name webserver -d nginx

# Port yonlendirme ile calistir (host:container)
docker run -d -p 8080:80 nginx

# Interaktif modda calistir (terminal erisimi ile)
docker run -it ubuntu bash

# Konteyner durdugunda otomatik sil
docker run --rm nginx

# Ortam degiskeni ile calistir
docker run -e MYSQL_ROOT_PASSWORD=sifre -d mysql
```

Konteynerleri Listeleme

```
# Calisan konteynerleri listele
docker ps

# Tum konteynerleri listele (calisan ve duran)
docker ps -a

# Kisa format
docker ps -q
```

Konteyner Baslatma/Durdurma/Yeniden Baslatma

```
# Calisan konteyneri durdur
docker stop <konteyner-id-veya-isim>

# Durdurulmus konteyneri baslat
docker start <konteyner-id-veya-isim>

# Konteyneri yeniden baslat
docker restart <konteyner-id-veya-isim>

# Konteyneri zorla durdur
docker kill <konteyner-id-veya-isim>
```

Konteyner Silme

```
# Bir konteyneri sil (once durdurulmali)
docker rm <konteyner-id>

# Calisan konteyneri zorla sil
docker rm -f <konteyner-id>

# Tum durdurulmus konteynerleri sil
docker container prune
```

Konteyner Icin Faydali Komutlar

```
# Konteyner icinde komut calistir
docker exec -it <konteyner-id> bash

# Konteyner loglarini goruntule
docker logs <konteyner-id>

# Loglari takip et (tail -f gibi)
docker logs -f <konteyner-id>

# Konteyner ici surecleri listele
docker top <konteyner-id>

# Konteyner istatistiklerini gor
docker stats <konteyner-id>

# Konteyner bilgilerini goruntule
docker inspect <konteyner-id>
```

Tablo 5 Konteyner Yonetim Komutlari Ozeti

Komut	Aciklama
`docker run`	Yeni konteyner olustur ve calistir
`docker ps`	Calisan konteynerleri listele
`docker ps -a`	Tum konteynerleri listele
`docker start`	Durdurulmus konteyneri baslat
`docker stop`	Calisan konteyneri durdur
`docker restart`	Konteyneri yeniden baslat
`docker rm`	Konteyneri sil
`docker exec`	Calisan konteyner icinde komut calistir
`docker logs`	Konteyner loglarini goruntule
`docker inspect`	Konteyner detayli bilgilerini gor
`docker stats`	Konteyner kaynak kullanimini izle

4.2 Image Yonetim Komutlari

```
# Yerel imaj listesini goruntule
docker images

# Docker Hub'dan imaj indir
docker pull python:3.11-slim

# Yerel imaji Docker Hub'a yukle
docker push kullaniciadi/imaj:etiket

# Imaji sil
docker rmi nginx:latest

# Kullanilmayan imajlari temizle
docker image prune

# Imajin katmanlarini ve gecmisini gor
docker history nginx

# Imaj hakkinda detayli bilgi
docker inspect nginx
```

4.3 Ag ve Volume Komutlari

Ag (Network) Islemleri

```
# Docker aglarini listele
docker network ls

# Yeni bir ag olustur
docker network create mynetwork

# Konteyneri belirli bir aga bagla
docker run -d --network mynetwork --name db mysql
docker run -d --network mynetwork --name web nginx

# Ag detaylarini goruntule
docker inspect mynetwork

# Agi sil
docker network rm mynetwork
```

Volume (Hacim) Islemleri

```
# Volume listesini goruntule
docker volume ls

# Yeni volume olustur
docker volume create mydata

# Konteyner ile volume bagla
docker run -d -v mydata:/var/lib/mysql mysql

# Host dizinini konteynere bagla (bind mount)
docker run -d -v /host/yol:/container/yol nginx

# Volume sil
docker volume rm mydata

# Kullanilmayan volumeleri temizle
docker volume prune
```

> **Volume ve Bind Mount Farki**
>
> **Volume:** Docker tarafindan yonetilen kalici depolama alanidir. Docker'in dosya sisteminde saklanir.
>
> **Bind Mount:** Host makinenin belirli bir dizininin konteynere baglanmasidir. Daha esnek ancak Docker disi yonetim gerektirir.

4.4 Docker Compose Temelleri

Docker Compose, cok konteynerli uygulamalari tanimlamak ve calistirmak icin kullanilan bir aractir. `docker-compose.yml` adli bir YAML dosyasi ile tum servislerinizi tanimlayabilir ve tek bir komutla calistirabilirsiniz.

Basit bir Docker Compose dosyasi ornegi:

```
version: '3.8'

services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html
    depends_on:
      - api

  api:
    image: python:3.11-slim
    working_dir: /app
    volumes:
      - ./api:/app
    command: python app.py

  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: sifre123
      MYSQL_DATABASE: mydb
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
```

Docker Compose Komutlari

```
# Tum servisleri baslat (arka planda)
docker compose up -d

# Tum servisleri durdur
docker compose down

# Servis loglarini gor
docker compose logs -f

# Belirli bir servisi yeniden baslat
docker compose restart api

# Servis olceklendir (ornegin 3 API instansi)
docker compose up -d --scale api=3
```

5. Dockerfile ile Uygulama Olusturma

> *Bu bolumde teorik bilgileri pratige dokerek gercek uygulamalar icin Dockerfile yazmayi ogreneceksiniz. Nginx web sunucusu ve Python web uygulamasi ornekleri ile adim adim ilerleyecegiz.*

5.1 Dockerfile Nedir?

Dockerfile, Docker imajlarini otomatik olarak olusturmak icin kullanilan bir metin dosyasidir. Icerisinde imajin nasil olusturulacagini belirten talimatlar (instructions) bulunur. Docker, bu dosyayi okuyarak adim adim bir imaj insa eder (build).

Dockerfile'in temel yapisini anlamak icin en sik kullanilan talimatlara bakalim:

Tablo 6 Dockerfile Temel Talimatlari

Talimat	Aciklama	Ornek Kullanim
`FROM`	Temel imaji belirler (her Dockerfile'da ilk satir)	`FROM ubuntu:22.04`
`RUN`	Imaj olusturulurken calistirilacak komut	`RUN apt-get update`
`COPY`	Host'tan imaja dosya kopyalar	`COPY . /app`
`ADD`	COPY gibidir, URL ve otomatik cikarma destegi var	`ADD https://ornek.com/dosya.tar.gz /app`
`WORKDIR`	Calisma dizinini ayarlar	`WORKDIR /app`
`ENV`	Ortam degiskeni tanimlar	`ENV NODE_ENV=production`
`EXPOSE`	Konteynerin hangi portu dinleyecegini bildirir	`EXPOSE 8080`
`CMD`	Konteyner calistiginda varsayilan komut	`CMD ["python", "app.py"]`

`ENTRYPOINT`	Konteyner calistiginda calisacak ana komut	`ENTRYPOINT ["python"]`
`VOLUME`	Kalici depolama alani tanimlar	`VOLUME ["/data"]`
`USER`	Konteynerin hangi kullaniciyla calisacagini belirler	`USER appuser`
`LABEL`	Imaja metadata ekler	`LABEL version="1.0"`

> **CMD ve ENTRYPOINT Farki**
>
> `ENTRYPOINT` sabit bir komut tanimlar ve genellikle degismez. `CMD` ise varsayilan parametreler saglar ve calisma zamaninda ezilebilir.
>
> Ornegin `ENTRYPOINT ["python"]` ve `CMD ["app.py"]` tanimliysa, konteyner calistiginda `python app.py` calisir. Ama `docker run imaj dosya.py` derseniz `python dosya.py` calisir.

5.2 Nginx Web Sunucusu Ornegi

Ilk ornegimizde, statik bir web sitesi sunan ozellestirilmis bir Nginx imaji olusturacagiz. Bu, Docker ogrenmek icin en basit ve en yaygin orneklerden biridir.

Proje Yapisi

```
nginx-ornek/
|-- Dockerfile
|-- html/
|   |-- index.html
|   |-- style.css
|-- nginx.conf
```

Adim 1: HTML Dosyalarini Olusturma

`html/index.html` dosyasi:

```
<!DOCTYPE html>
<html lang="tr">
<head>
    <meta charset="UTF-8">
    <title>Docker Nginx Ornegi</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h1>Merhaba Docker!</h1>
        <p>Bu sayfa Docker konteyneri icinde calisan Nginx tarafindan sunuluyor.
</p>
        <div class="badge">Docker + Nginx</div>
    </div>
</body>
</html>
```

`html/style.css` dosyasi:

```
body {
    font-family: Arial, sans-serif;
    background: #f0f4f8;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
}
.container {
    text-align: center;
    background: white;
    padding: 50px;
    border-radius: 10px;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}
h1 { color: #2c5282; }
.badge {
    display: inline-block;
    margin-top: 20px;
    padding: 10px 20px;
    background: #4299e1;
    color: white;
    border-radius: 20px;
}
```

Adim 2: Nginx Konfigurasyonu

`nginx.conf` dosyasi:

```
server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Adim 3: Dockerfile Yazma

`Dockerfile` dosyasi:

```
# Temel imaj olarak resmi Nginx imajini kullan
FROM nginx:alpine

# Olusturan bilgisi (opsiyonel)
LABEL maintainer="ornek@mail.com"
LABEL description="Statik web sitesi sunan Nginx konteyneri"

# Ozellestirilmis nginx konfigurasyonunu kopyala
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Statik dosyalari Nginx'in varsayilan dizinine kopyala
COPY html/ /usr/share/nginx/html/

# 80 numarali portu disari ac
EXPOSE 80

# Nginx'i on plan calistir
CMD ["nginx", "-g", "daemon off;"]
```

Adim 4: Image Olusturma ve Calistirma

```
# Image'i olustur (proje dizininde calistir)
docker build -t benim-nginx:1.0 .

# Image'in olustugunu kontrol et
docker images | grep benim-nginx

# Konteyneri calistir
docker run -d --name webserver -p 8080:80 benim-nginx:1.0

# Kontrol et
docker ps

# Tarayicida ac: http://localhost:8080
```

1 **Image'i Olustur**

`docker build` komutu, Dockerfile'dan imaj olusturur. `-t` parametresi imaja isim ve etiket verir, `.` ise Dockerfile'in bulundugu dizini gosterir.

2 **Konteyneri Calistir**

`-p 8080:80` ile host makinenin 8080 portunu konteynerin 80 portuna yonlendiriyoruz. Boylece tarayicidan erisim saglanabilir.

3 **Test Et**

Tarayicida `http://localhost:8080` adresine giderek statik web sitenizi goruntuleyebilirsiniz.

5.3 Python Web Uygulamasi Ornegi

Simdi daha gercekci bir ornek ile Python Flask web uygulamasi konteynerlestirecegiz. Bu ornek, bagimlilik yonetimi ve cevre degiskeni kullanimini gostermektedir.

Proje Yapisi

```
python-ornek/
|-- Dockerfile
|-- requirements.txt
|-- app.py
|-- templates/
|   |-- index.html
```

Adim 1: Python Uygulamasi

`app.py` dosyasi:

```
from flask import Flask, render_template
import os

app = Flask(__name__)

@app.route('/')
def hello():
    isim = os.environ.get('KULLANICI_ADI', 'Dunya')
    return render_template('index.html', isim=isim)

@app.route('/durum')
def durum():
    return {'durum': 'calisiyor', 'servis': 'python-web'}

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

`requirements.txt` dosyasi:

```
Flask==3.0.0
Werkzeug==3.0.1
```

Adim 2: Dockerfile

```
# Python 3.11 slim temel imaji
FROM python:3.11-slim

# Guvenlik icin root disinda kullanici olustur
RUN useradd -m -u 1000 appuser

# Calisma dizini olustur
WORKDIR /app

# Bagimliliklari once kopyala (cache icin)
COPY requirements.txt .

# Python bagimliliklarini yukle
RUN pip install --no-cache-dir -r requirements.txt

# Uygulama kodunu kopyala
COPY app.py .
COPY templates/ ./templates/

# Kullaniciya gec
USER appuser

# Ortam degiskeni
ENV KULLANICI_ADI=Docker

# Port ac
EXPOSE 5000

# Uygulamayi baslat
CMD ["python", "app.py"]
```

Adim 3: Build ve Run

```
# Image olustur
docker build -t python-web:1.0 .

# Calistir (varsayilan kullanici adi ile)
docker run -d --name pyapp -p 5000:5000 python-web:1.0

# Calistir (ozel kullanici adi ile)
docker run -d --name pyapp2 -p 5001:5000 \
  -e KULLANICI_ADI=Ahmet python-web:1.0

# Loglari kontrol et
docker logs pyapp

# Test et
curl http://localhost:5000
curl http://localhost:5000/durum
```

Dockerfile En Iyi Pratikleri

- **Kucuk temel imajlar kullanin:** `python:3.11-slim` yerine `python:3.11` yerine slim veya alpine varyantlarini tercih edin.
- **Bagimliliklari once kopyalayin:** Bu, Docker cache mekanizmasindan yararlanmanizi saglar.
- **Root disinda kullanici calistirin:** Guvenlik icin konteyneri root yetkisi olmayan bir kullanici ile calistirin.
- **.dockerignore kullanin:** Gereksiz dosyalarin imaja katilmasini onleyin.

5.4 Multi-Stage Build

Multi-Stage Build, buyuk uygulamalar icin imaj boyutunu optimize etmek amaciyla kullanilan gelismis bir Dockerfile teknigidir. Bu yontemde, uygulama derleme (build) ve calistirma (runtime) asamalari ayri imajlarda gerceklesir.

Ornek olarak, bir Go uygulamasi derleyelim:

```
# ========== ASAMA 1: Build ==========
FROM golang:1.21 AS builder

WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download

COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o myapp .

# ========== ASAMA 2: Runtime ==========
FROM alpine:latest

WORKDIR /app

# Sadece derlenmis binary'i kopyala
COPY --from=builder /app/myapp .

EXPOSE 8080
CMD ["./myapp"]
```

Bu yaklasimin avantajlari:

- **Daha kucuk imaj:** 500 MB'lik build imaji yerine 20 MB'lik runtime imaji
- **Daha guvenli:** Derleme araclari runtime imajinda bulunmaz
- **Daha hizli dagitim:** Kucuk imajlar daha hizli yuklenir ve dagitilir

> **Boyut Karsilastirmasi**
>
> Ayni uygulama icin:
>
> **Single-stage:** Golang imaji + uygulama = ~800 MB
> **Multi-stage:** Alpine + sadece binary = ~15 MB
> **Tasarruf:** Yaklasik %98 boyut azalmasi!

6. Sonuc ve Oneriler

> *Bu el kitabi boyunca Docker'in temel kavramlarini, mimarisini, temel komutlarini ve pratik uygulamalarini ogrendik. Bu bolumde ogrendiklerimizi ozetleyecek ve bir sonraki adimlar icin oneriler sunacagiz.*

Ogrenilenlerin Ozeti

Bu rehberde asagidaki konulari ele aldik:

1. **Docker Nedir:** Konteynerlestirme teknolojisinin temel kavramlarini ve Docker'in sagladigi avantajlari ogrendik.
2. **VM Karsilastirmasi:** Sanal makineler ile Docker konteynerleri arasindaki farklari, avantaj ve dezavantajlari inceledik.
3. **Docker Mimarisi:** Docker Engine, Image, Container, Docker Hub ve diger temel bilesenleri detayli olarak ogrendik.
4. **Temel Komutlar:** Konteyner ve imaj yonetimi, ag ve volume islemleri, Docker Compose kullanimini ogrendik.
5. **Dockerfile:** Gercek uygulamalari konteynerlestirmek icin Dockerfile yazmayi, Nginx ve Python ornekleri uzerinden pratik yaptik.

Bir Sonraki Adimlar

Docker temellerini ogrendikten sonra asagidaki konulari ogrenebilirsiniz:

1. Docker Compose Ileri Seviye

- Coklu ortam yapilandirmalari (development, staging, production)
- Health check ve restart politikalarini anlama
- Compose override dosyalari ile esneklik

2. Docker Swarm veya Kubernetes

- **Docker Swarm:** Docker'in yerlesik konteyner orkestrasyon araci

- **Kubernetes:** Endustrinin en yaygin kullanilan konteyner orkestrasyon platformu

3. CI/CD Entegrasyonu

- Jenkins, GitLab CI, GitHub Actions ile otomatik image olusturma
- Otomatik test ve dagitim pipeline'lari kurma

4. Guvenlik ve Izlenebilirlik

- Docker imaj guvenlik taramasi (Trivy, Clair)
- Konteyner loglama ve monitoring (Prometheus, Grafana)
- Docker imaj imzalama ve guvenli kayit defteri kullanimi

Kaynaklar

- **Docker Resmi Dokumantasyonu:** https://docs.docker.com
- **Docker Hub:** https://hub.docker.com
- **Play with Docker:** https://labs.play-with-docker.com (Ucretsiz Docker ogrenme ortami)
- **Docker CLI Referansi:** https://docs.docker.com/engine/reference/commandline/cli/

> **Alinacak Etkinlikler**
>
> - Play with Docker uzerinde ucretsiz olarak pratik yapin
> - Kendi projelerinizi konteynerlestirmeye calisin
> - Docker Hub'da populer imajlari inceleyin
> - Dockerfile'larinizi optimize etmek icin multi-stage build tekniklerini uygulayin

Bu rehber, Docker ogrenme yolculugunuzda ilk adimi atmak icin hazirlanmistir.
Iyi calismalar!